

Task 1:

1.5)

An if/else chain would break the open-closed principle as the principle states that classes should be extended but not modified. Any time a new product and corresponding factory are added, the code of the createConnector function would always need to be changed which could slow down our program. Also we would need to test the createConnector function every time to see if nothing broke with the additional changes. The Factory design pattern correctly follows the open-close principle as when a new product and its corresponding factory are added the Connector interface is still able to call to that product, all that differs is that the new factory needs to implement createConnector function for its corresponding product. The factory method allows you to add new features by simply adding in new classes, keeping to the open-close principle.

1.6)

Connector - Acts as an interface to access the concrete product, itself is an abstract product

CsvConnector - Concrete product

ConnectorFactory - Abstract factory, interface

CsvFactory - concrete factory

Task 2:

2.5)

A shallow copy copies an object's references so the values that the variables in the new object are referencing in memory are the same ones that the original object references whereas, a deep copy makes a completely independent copy of an object. An object that has been deep copied has its own memory allocated to store its values. The Clone() method in the Transformation class creates deep copies. The clone must be independent of the prototype so that anything applied to a pipeline doesn't affect other pipelines that may be using the same type of prototype.

2.6)

The registry returns clones instead of pointers to the stored prototypes so that every pipeline gets its own new and isolated instance of a Transformation and if two pipelines shared one prototype, one pipeline could unknowingly change the state of the other pipeline which could lead to incorrect output and also if the one pipeline is complete and deletes the shared prototype, when the other pipeline tries to access it it will lead to a segmentation fault.

Task 3:

3.5) Run is not virtual because it needs every pipeline to use it to make sure all pipelines connect, extract, load and transform. If it was virtual and a subclass was able to override the run method then it could break the deliberate order that run calls the methods inside it or it could leave out one of the methods it's meant to call.

3.6)

Run is the template method, Connect is a concrete step, Transform is a concrete step, Extract and Load are primitive operations. The Factory method in pipeline lets pipeline have a dedicated factory for creating connectors (createConnector) which oversees how

connectors are created while the template method makes a standard way for all pipelines to execute their methods by controlling the steps that take place and in what order they happen and how some of the steps can differ. These do not overlap because they do not address the same areas in the code. The Factory method looks at the creation of the connectors and the template method looks at the way the pipelines should behave (how the steps should execute and when they should execute).

Task 4:

4.4)

The narrow interface of the Memento (RunCheckpoint) keeps its internal state private and immutable from the Caretaker(CheckpointManager) but gives it the functionality to store and move the memento. The wide interface is used by the Originator (Pipeline). This wide interface exposes the state of the memento to the originator only and lets it set and read the state.

4.5)

The pipeline takes snapshots as it goes through the various steps of the running process. The CreateCheckpoint function is used to create a RunCheckpoint that stores the state of that snapshot. RunCheckpoint object is then transferred to the CheckpointManager and the save function is used to internally store the snapshot into the Caretaker's memory (history). When the load process crashes the undo function is called which removes and returns the most recent snapshot (RunCheckpoint) in memory. This snapshot is then passed to the Pipeline with the restore function and the records and stage are reset to the stage and records of the passed in RunCheckpoint object.

4.6)

Pipeline – The originator and has its own internal state of stages and records which are used to create mementos.

RunCheckpoint - It is the Memento and creates a snapshot of the pipeline at a certain phase. It records its state (the stage and records) and allows the Originator to have access to this state further down the line. In addition, it provides a narrow interface to the Caretaker (CheckpointManager) keeping its state private to outside classes but visible to the Originator with its wide interface.

CheckpointManager - It is the Caretaker. It has no grasp of the properties of the snapshots (the RunCheckpoint objects) but through the Memento's narrow interface it can store it in its memory. This enables different states of the Pipeline to be stored and accessed for future referencing for when the system potentially crashes during a run.